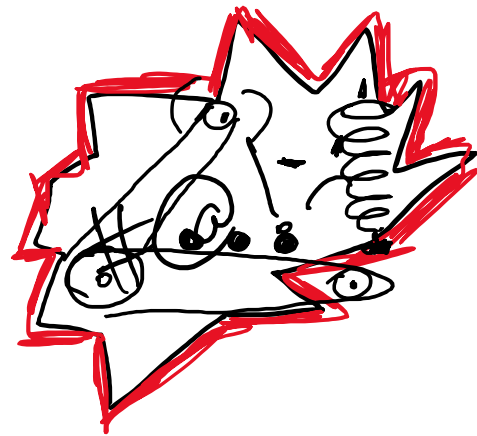
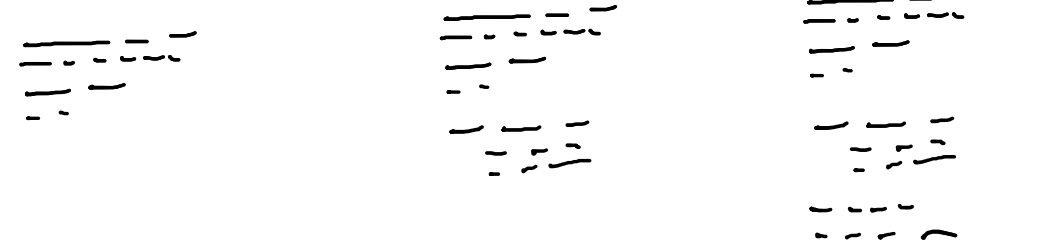


code ... code ... code ...



~~Never change a running system~~

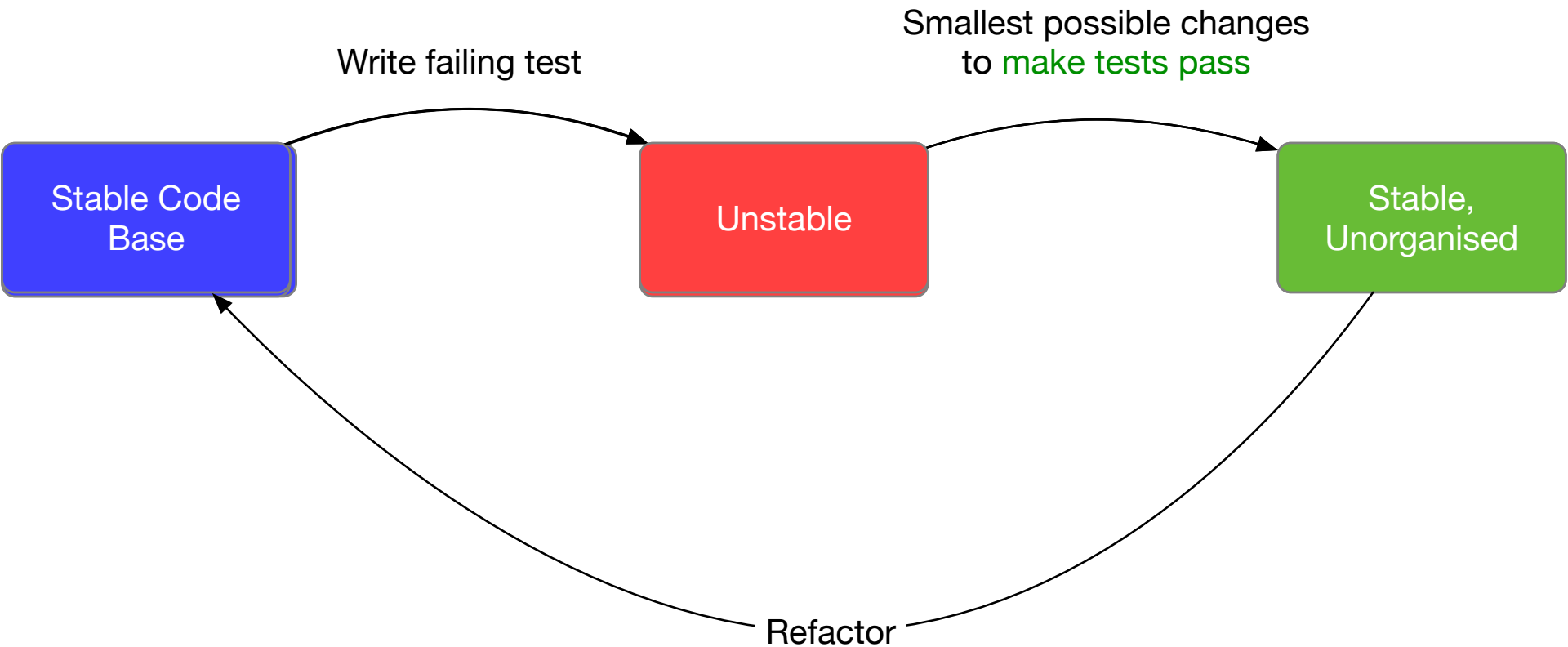
change is inevitable

Testing

- "Testing shows the presence, not the absence of bugs" (Edsger W Dijkstra)
- Test to get confidence in software meeting requirements
- Test to show new changes don't break existing code (regression testing)

TDD

- XP
- How do we build software that is easy to test
- More than test-first development



TDD main steps

- Write test based on user stories or detailed requirements
 - focus on the requirements *before* writing the code
- Focus implementation on the essentials
 - Don't pre-maturely optimize
- Re-run tests
- When all tests pass: Refactor
 - Restructure, rename, remove duplication
 - Break functions, objects into more manageable chunks
 - Introduce interfaces, inheritance, new modules
 - Keep running tests
 - Refactor tests

Handling Dependencies During Testing

- Dummy
 - Objects that are not used
 - Indicates code smell
- Stub
 - Canned, static response
 - Time-service that returns always same time
- Fake
 - Working implementation that is not fit for production use
 - In-memory DB
- Mock
 - Object that tracks calls by the tested code and raises exceptions if not met
 - MockWebService that expects a call to .service by your code

Limitations

- Tests might be incomplete and the implementation duplicating errors in the tests
- Tests need to be accompanied with code coverage reports
- If tests fail, it is important to fix them
- Maintaining tests can be expensive